

编 号	题 号	标 题	知 识 点	代 码 作 者
例 2-15	POJ 1833	排列	下一个排列	陈锋
例 2-16	POJ 1731	Orders	生成可重集的排列	陈锋
例 2-17	UVa 816	Abbott's Revenge	BFS 搜索最短路径	刘汝佳
例 2-18	UVa 1601	The Morning after Halloween	BFS: 路径查找优化	陈锋
例 2-19	POJ 1915	Knight Moves	双向 BFS	陈锋
例 2-20	UVa 167	The Sultan's Successors	八皇后问题	刘汝佳
例 2-21	UVa 524	Prime Ring Problem	回溯法和生成测试法的比较	陈锋
例 2-22	UVa 129	Krypton Factor	回溯法: 避免无用判断	陈锋
例 2-23	BIO 1997 Round 1 Question 3	Egyptian Fractions	迭代加深搜索	陈锋
例 2-24	UVa 12558	Egyptian Fractions	IDA*: 迭代加深搜索	陈锋
例 2-25	POJ 2449	Remmarguts' Date	A*: K 短路问题	陈锋
例 2-26	UVa 11214	Guarding the Chessboard	棋盘皇后覆盖	陈锋
例 2-27	UVa 1085	House of Cards	极大极小过程: Alpha-beta 剪枝: 对抗搜索	刘汝佳
例 2-28	POJ 3076	Sudoku	数独: Dancing Links	刘汝佳
例 2-29	NC 50517	数字游戏	数位统计: DFS	陈锋
例 2-30	NC 50516	Amount of Degrees	数位统计: DFS	陈锋

2.4 动态规划算法

几乎所有算法竞赛中都会出现考察动态规划算法的题目。本节主要介绍一些经典问题（如LIS、LCS、最优矩阵链乘、树的重心和TSP等）的代码实现。

例2-31 【01背包；滚动数组优化】手链（Charm Bracelet, POJ 3624）

给出 N ($1 \leq N \leq 3402$) 个珠子，其中第 i 个珠子的重量是 W_i ($1 \leq W_i \leq 400$)，价值是 D_i ($1 \leq D_i \leq 100$)。现在要从中挑出尽量多的珠子，在总重量不超过 M ($1 \leq M \leq 12\ 880$) 的情况下，使得总价值最大。计算总价值的最大值。

【代码实现】

```

// 陈锋
#include <algorithm>
#include <cmath>
#include <cstdio>

const int MAXN = 5000;
using namespace std;
int N, M, W[MAXN], D[MAXN], S[20 * MAXN];
int main() {
    scanf("%d %d", &N, &M); // n为商品数量, m为背包大小
    for (int i = 1; i <= N; i++) scanf("%d %d", &W[i], &D[i]);

    // 二维背包空间  $O(N \times M)$ , 对于本题而言为  $3402 \times 12880$ , 所以空间太大
    // 采用滚动数组, 空间降为  $O(M)$ 
    //  $S(i, j)$ : 前  $i$  个珠子, 总重不超过  $j$  所能获得的最大价值
    //  $S(i, j) = \max(S(i-1, j), S(i-1, j-W(i)) + D[i])$ 
    // 滚动数组, 仅保留第 2 维度,  $S[j] = \max(S[j], S[j-W[i]] + D[i])$ 
    // 按照从大到小的顺序更新数组
    for (int i = 1; i <= N; i++)
        for (int j = M; j >= W[i]; j--) S[j] = max(S[j], S[j - W[i]] + D[i]);
    printf("%d\n", S[M]);
    return 0;
}

/*
同类题目: Jin Ge Jin Qu [h]ao, UVa 12563
          Team them up!, UVa 1627
          Bookcase, UVa 12099
*/

```

例2-32 【树的最大独立集】Hali-Bula的晚会 (Party at Hali-Bula, UVa 1220)

公司里有 n ($n \leq 200$) 个人形成一个树状结构, 即除老板外, 每名员工都有唯一的直属上司。要求选尽量多的人, 但不能同时选择一个人和他的直属上司。问: 最多能选多少人? 在人数最多的前提下方案是否唯一?

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
const int MAXN = 200 + 4;
```

```

vector<int> CH[MAXN];
int D[MAXN][2][2], VIS[MAXN][2];
// 子树 u, 父结点选中状态, 父结点 pa
int dp(const int u, const int pa) {
    assert(pa == 0 || pa == 1);
    int &d = D[u][pa][0], &cnt = D[u][pa][1]; // 最大值, 方案数
    if (VIS[u][pa]) return d;
    VIS[u][pa] = 1;
    d = 0, cnt = 1;
    for (size_t i = 0; i < CH[u].size(); i++)
        d += dp(CH[u][i], 0), cnt *= D[CH[u][i]][0][1];

    if (pa == 0) { // u 的父亲未选
        int nd = 1, c = 1; // 选 u 的对应结果
        for (size_t i = 0; i < CH[u].size(); i++)
            nd += dp(CH[u][i], 1), c *= D[CH[u][i]][1][1]; // 子树方案数之积
        if (nd == d) cnt += c;
        else if (nd > d) d = nd, cnt = c;
    }
    return d;
}

map<string, int> names;
int nCnt = 0;
int getId(const string& s) {
    if (!names.count(s)) names[s] = nCnt++;
    return names[s];
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    string n1, n2;
    for (int N; cin >> N && N; ) {
        for (int i = 0; i < N; i++) CH[i].clear();
        names.clear(), nCnt = 0;
        memset(D, 0, sizeof(D)), memset(VIS, 0, sizeof(VIS));
        cin >> n1, names[n1] = nCnt++;
        for (int i = 1; i < N; i++) {
            cin >> n1 >> n2;
            int pa = getId(n2), u = getId(n1);
            CH[pa].push_back(u);
        }
        int ans = dp(0, 0);
        printf("%d %s\n", ans, (D[0][0][1] == 1 ? "Yes" : "No"));
    }
    return 0;
}
/*

```

算法分析请参考:《算法竞赛入门经典(第2版)》例 9-13

```
同类题目: Another Crisis, UVa 12186
          Perfect Service, ACM/ICPC Kaoshiung 2006, UVa 1218
          骑士, ZJOI 2008, 牛客NC 20481
*/
```

例2-33 【最长公共子序列问题 (LCS)】 公共子序列 (Common Subsequence, POJ 1458)

给出一个序列，从中删除零到多个元素后，剩余的就是其子序列。例如， $Z=\{a, b, f, c\}$ 是 $X=\{a, b, c, f, b, c\}$ 的子序列，索引序列为 $\{1, 2, 4, 6\}$ 。给定两个序列 X 和 Y ，计算其最大长度公共子序列的长度。

【代码实现】

```
// 陈锋
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstring>
const int NN = 1000;
using namespace std;
char s1[NN], s2[NN];
int dp[NN][NN];
int main() {
    while (scanf("%s %s", s1, s2) == 2) {
        int L1 = strlen(s1), L2 = strlen(s2);
        memset(dp, 0, sizeof(dp));
        for (int i = 0; i < L1; i++)
            for (int j = 0; j < L2; j++)
                if (s1[i] == s2[j]) // 相同字符
                    dp[i + 1][j + 1] = dp[i][j] + 1;
                else // 在两个序列中选择一个删除字符
                    dp[i + 1][j + 1] = max(dp[i][j + 1], dp[i + 1][j]);
        printf("%d\n", dp[L1][L2]);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典 (第2版)》9.4.1 节最长公共子序列问题 (LCS)
同类题目: Cyborg Genes, UVa 10723
*/
```

例2-34 【最长上升子序列问题 (LIS)】 对齐 (Alignment, POJ 1836)

给定 n 个整数 $A_1, A_2, \dots, A_n$ ，按从左到右的顺序选出尽量多的整数，组成一个先升后降的子序列（子序列可以理解为：删除0个或多个数后，其他数的顺序不变）。例如，对于序列{1, 6, 2, 3, 7, 5}，可以选出上升子序列{1, 2, 3, 5}，也可以选出上升子序列{1, 6, 7}，但前者更长。选出的子序列中相邻元素不能相等。

【代码实现】

```

// 陈锋
#include <algorithm>
#include <cstdio>
#include <cstring>
const int NN = 1000 + 4;
using namespace std;
int F[NN], G[NN];
float A[NN];
int main() {
    for (int n, ans = 0; scanf("%d", &n) == 1;) {
        fill_n(F, n, 0), fill_n(G, n, 0);
        for (int i = 1; i <= n; i++) scanf("%f", &A[i]);
        for (int i = 1; i <= n; i++) {
            F[i] = 1; // F[i]: 以 i 为结尾的最长 LIS
            for (int j = i - 1; j >= 1; j--)
                if (A[i] > A[j] && F[i] < F[j] + 1) F[i] = F[j] + 1;
        }

        for (int i = n; i >= 1; i--) {
            G[i] = 1; // G[i]: 以 i 为开始的最长 LDS
            for (int j = i + 1; j <= n; j++)
                if (A[i] > A[j] && G[i] < G[j] + 1) G[i] = G[j] + 1;
        }

        for (int i = 1; i <= n; i++)
            for (int j = i + 1; j <= n; j++) ans = max(ans, F[i] + G[j]);
        printf("%d\n", n - ans);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》9.4.1 节中最长上升子序列问题
同类题目: Constructing Roads In JGShining's Kingdom, HDU 1025
*/

```

例 2-35 【LCS; 可转化为 LIS】王子和公主 (Prince and Princess, UVa 10635)

有两个长度分别为 $p+1$ 和 $q+1$ 的序列 A 和 B , 每个序列中的各个元素互不相同, 且都是 $1 \sim n^2$ ($2 \leq n \leq 250$, $1 \leq p, q \leq n^2$) 的整数。两个序列的第一个元素均为 1。求出 A 和 B 的最长公共子序列长度。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 256, MAXP = MAXN * MAXN;
typedef vector<int> VI;
int IDX[MAXN];
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T; cin >> T;
    for (int t = 1, n, a, p, q; cin >> n >> p >> q && n; t++) {
        ++p, ++q;
        fill_n(IDX, n * n + 2, 0);
        for (int i = 1; i <= p; i++) cin >> a, IDX[a] = i;
        VI D;
        for (int i = 0, b; i < q; i++) {
            cin >> b, b = IDX[b];
            if (b == 0) continue;
            VI::iterator it = lower_bound(D.begin(), D.end(), b);
            if (it == D.end()) D.push_back(b);
            else *it = b;
        }
        printf("Case %d: %llu\n", t, D.size());
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 1.4 节例题 27
*/

```

例2-36 【最大连续和】最大连续子序列 (HDU 1231)

给定 K 个整数的序列 $\{N_1, N_2, \dots, N_K\}$, 其任意连续子序列可表示为 $\{N_i, N_{i+1}, \dots, N_j\}$, 其中 $1 \leq i \leq j \leq K$ 。最大连续子序列是所有连续子序列中元素和最大的一个, 如给定序列 $\{-2, 11, -4, 13, -5, -2\}$, 其最大连续子序列为 $\{11, -4, 13\}$, 最大和为20。

计算最大和的子序列, 输出其元素和以及第一个和最后一个元素。

【代码实现】


```

// 陈锋
#include <bits/stdc++.h>
/*
d(i)表示以 i 为结尾的最大连续和, d(i)=max(0,d(i-1))+Ai
要么包含 D[i-1], 要么不包含
*/
using namespace std;
const int NN = 10004;
int D[NN], A[NN];
int main() {
    for (int n; scanf("%d", &n) == 1 && n;) {
        D[0] = 0;
        for (int i = 1; i <= n; i++) {
            int &a = A[i];
            scanf("%d", &a);          // D[i]: 以 i 为结尾的子序列和的最大值
            D[i] = max(D[i - 1] + a, a); // 是否包含 A[i]
        }
        int ed_v = A[1], maxd = D[1], ed = 1;
        for (int i = 1; i <= n; i++) // 最大子序列结束元素及其位置
            if (D[i] > maxd) maxd = D[i], ed_v = A[i], ed = i;
        int st_v = A[ed]; // 开始元素值
        for (int i = ed, s = 0; i > 0 && s < maxd; i--) s += A[i], st_v = A[i];
        if (maxd >= 0) printf("%d %d %d\n", maxd, st_v, ed_v);
        else printf("%d %d %d\n", 0, A[1], A[n]);
    }
    return 0;
}
/*
同类题目: Max Sum, HDU 1003
*/

```

例2-37 【矩阵链乘】乘法谜题（Multiplication Puzzle, POJ 1651）

乘法谜题是用一行纸牌玩的游戏，每张纸牌包含一个正整数。在游戏过程中，玩家将一张纸牌从该行中取出，并为其打分，分值等于所获得的纸牌所含整数与该牌左右两侧纸牌所含整数的乘积。不允许取出该行中的第一张和最后一张牌。最后一步之后，该行只剩下两张牌。

计算最终总得分的最小值。

例如，假设该行中的纸牌包含数字10、1、50、20、5，则玩家可能会依次选择一张所含整数为1、20和50的纸牌，该做法最终所得分数为 $10 * 1 * 50 + 50 * 20 * 5 + 10 * 50 * 5 = 500 + 5000 + 2500 = 8000$ 。如果他以相反的顺序拿牌，即50、20，然后1，则得分为 $1 * 50 * 20 + 1 * 20 * 5 + 10 * 1 * 5 = 1000 + 100 + 50 = 1150$ 。

【代码实现】

```
// 陈锋
#include <algorithm>
#include <cstdio>
#include <cstring>
using namespace std;
const int NN = 100 + 10;
int N, A[NN], F[NN][NN]; // DP数组，F[i][j]表示取出 i~j 数字的最小花费

int main() {
    scanf("%d", &N);
    // 注意：memset 为 0x3f 会使得每个数组中的数字都为 0x3f3f3f3f，是一个极大值
    memset(F, 0x3f, sizeof F);
    for (int i = 1; i <= N; i++) scanf("%d", A + i);
    for (int len = 2; len <= N; len++) // 枚举长度
        for (int i = 1; i + len - 1 <= N; i++) { // 枚举起始点
            F[i][i] = F[i][i + 1] = 0;
            for (int j = i + len - 1, k = i; k < j; k++)
                F[i][j] = min(F[i][k] + F[k][j] + A[i] * A[k] * A[j], F[i][j]);
        }
    printf("%d", F[1][N]);
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》9.4.1节最优矩阵链乘
同类题目：Expression, HDU 5396
*/
```

例2-38 【区间切割DP】切木棍（Cutting Sticks, UVa 10003）

有一根长度为 L ($L < 1000$) 的木棍，还有 n ($n < 50$) 个切割点的位置（按照从小到大排列）。你的任务是在这些切割点的位置处把

棍子切成 $n+1$ 部分，使得总切割费用最小。每次切割的费用等于被切割的木棍长度。例如， $L=10$ ，切割点为2, 4, 7。如果按照2, 4, 7的顺序切，费用为 $10+8+6=24$ ；如果按照4, 2, 7的顺序切，费用为 $10+4+6=20$ 。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
const int MAXN = 50 + 2, INF = 0x7f7f7f7f;
int L, N, C[MAXN], D[MAXN][MAXN];

int dp(int l, int r) { // O(n^3)
    assert(l < r);
    int &d = D[l][r];
    if (d != -1) return d;
    if (l + 1 == r) return d = 0;
    d = INF;
    for (int i = l + 1; i < r; i++) // 选择一个点作为切割点
        d = min(C[r] - C[l] + dp(l, i) + dp(i, r), d);
    return d;
}

int main() {
    while (scanf("%d", &L) == 1 && L) {
        scanf("%d", &N);
        C[0] = 0, C[N + 1] = L;
        for(int i = 1; i <= N; i++) scanf("%d", &(C[i]));
        memset(D, -1, sizeof(D));
        int ans = dp(0, N + 1);
        printf("The minimum cutting is %d.\n", ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 9-9
同类题目：Cake Slicing, ACM/ICPC Nanjing 2007, UVa 1629
          棋盘分割, POJ 1191
          Post Office, POJ 1161
*/
```

例2-39 【括号序列】 括号序列 (Brackets Sequence, NEERC 2001, UVa 1626)

定义如下正规括号序列 (字符串)。

- ☐ 空序列是正规括号序列。
- ☐ 如果 S 是正规括号序列, 那么 (S) 和 $[S]$ 也是正规括号序列。
- ☐ 如果 A 和 B 都是正规括号序列, 那么 AB 也是正规括号序列。

例如, 下面的字符串都是正规括号序列: $()$, $[]$, $(())$, $([])$, $()[]$, $()[]()$; 而如下字符串则不是正规括号序列: $($, $[$, $]$, $)$, $((()$ 。

输入一个长度不超过100, 由“(” “)” “[” “]” 构成的序列, 添加尽量少的括号, 得到一个正规括号序列。如有多解, 输出任意一个序列即可。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;

const int MAXN = 100 + 4, INF = 0x7f7f7f7f;
char S[MAXN];
map<char, string> M2;

inline bool match(char c1, char c2) {
    return (c1 == '[' && c2 == ']') || (c1 == '(' && c2 == ')');
}

int D[MAXN][MAXN], N;
void init_dp() { // (i, j) <- i+1, j-1, i+1, j-1, (i, k + k+1, j)
    memset(D, 0, sizeof(D));
    _rep(i, 0, N - 1) D[i][i] = 2;
    for (int i = N - 2; i >= 0; i--) {
        _rep(j, i + 1, N - 1) {
            int& d = D[i][j];
            d = INF;
            if (match(S[i], S[j])) d = min(d, 2 + D[i + 1][j - 1]);
            _for(m, i, j) d = min(d, D[i][m] + D[m + 1][j]);
        }
    }
}

ostream& print(ostream& os, int i, int j) {
    if (i > j) return os;
    char cl = S[i], cr = S[j];
    if (i == j) return os << M2[cl];
    int d = D[i][j];

    if (match(cl, cr) && d == D[i + 1][j - 1] + 2)
        return print(os << cl, i + 1, j - 1) << cr;
    _for(m, i, j) if (d == D[i][m] + D[m + 1][j])
        return print(print(os, i, m), m + 1, j);
    return os;
}

int main() {
    int T;
    M2['['] = "[]", M2['('] = "()", M2['{'] = "{}", M2['>'] = "<";
    fgets(S, MAXN, stdin);
    sscanf(S, "%d", &T);
    _for(t, 0, T) {
        if (t) puts("");
        fgets(S, MAXN, stdin), fgets(S, MAXN, stdin);
        N = strlen(S) - 1;
        init_dp();
        stringstream ss;
        print(ss, 0, N - 1);
        puts(ss.str().c_str());
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 9-10
注意：本题中对于 ostream& 类型的使用
*/

```

例 2-40 【字符串划分问题；回文预处理】划分成回文串
(Partitioning by Palindromes, UVa 11584)

输入一个由小写字母组成的字符串，你的任务是把它划分成尽量少的回文串。例如，racecar本身就是回文串；fastcar只能分成7个单字母的回文串；aaadbcccb最少可分成3个回文串aaa, d, bccb。字符串长度不超过1000。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
const int MAXN = 1024;
int N, isPa[MAXN][MAXN], D[MAXN];
string S;

int dp(int k) { // S[0,k]对应的方案数  $O(n^2)$ 
    int &d = D[k];
    if (d != -1) return d;
    d = MAXN;
    if (isPa[0][k]) return d = 1;
    for (int i = 1; i <= k; i++)
        if (isPa[i][k]) d = min(1 + dp(i - 1), d);
    return d;
}

int main() {
    int T;
    cin >> T;
    while (T--) {
        cin >> S, N = S.length();
        memset(isPa, 0, sizeof(isPa)), memset(D, -1, sizeof(D));
        for (int i = 0; i < N; i++) {
            int l = i, r = i;
            while (l >= 0 && r < N && S[l] == S[r]) isPa[l--][r++] = 1;
            l = i, r = i + 1;
            while (l >= 0 && r < N && S[l] == S[r]) isPa[l--][r++] = 1;
        }
        int ans = dp(N - 1);
        printf("%d\n", ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 9-7
注意：预处理所有回文的方法
同类题目：Chopsticks, UVa 10271
          Folding, NEERC 2001, UVa 1630
          Palindrome, POJ 1159
*/

```

例2-41 【树形DP；二次换根】计算机网络（Computer, HDU 2196）

给定一棵树，边有权值，输出离每个点最远的那个点的距离。

【代码实现】


```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
const int NN = 10000 + 8;
struct Edge {
    int v, w;
};
vector<Edge> G[NN];
int N, F[NN], SF[NN], FV[NN], H[NN];
/*
F(u): Tree(u)中距离u的最长距离; H(u): 树中除子树u外的点到u的最大距离。
F(u)一次DFS即可:  $F(u) = \max\{w(u \rightarrow v) + F(v)\}$ 
如何计算H? 假设u有k个子:  $v_1, v_2, \dots, v_k$ :
v不是F(u)经过的点,  $H(v) = w(v, u) + \max\{F(u), H(u)\}$ 
反之, 不能选择F(u), 要选择Tree(u)第二大距离SF(u),  $H(v) = w(v, u) + \max\{SF(u), H(u)\}$ 
所求结果就是  $\max\{F(u), H(u)\}$ 
*/
int dfsF(int u) {
    int &f = F[u], &sf = SF[u];
    f = 0, sf = 0;
    for (size_t ei = 0; ei < G[u].size(); ei++) {
        const Edge& e = G[u][ei];
        int nf = dfsF(e.v) + e.w;
        if (nf >= f) {
            sf = f;
            f = nf, FV[u] = e.v;
        } else if (nf > sf)
            sf = nf;
    }
    return f;
}

void dfsH(int u) { // H[u]: 所求解
    for (size_t ei = 0; ei < G[u].size(); ei++) {
        const Edge& e = G[u][ei];
        H[e.v] = max(H[u], FV[u] == e.v ? SF[u] : F[u]) + e.w;
        dfsH(e.v);
    }
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    while (cin >> N && N) {
        for (int i = 1; i <= N; ++i) G[i].clear();
        for (int u = 2, v, w; u <= N; ++u)
            cin >> v >> w, G[v].push_back((Edge){u, w});
        dfsF(1);
        H[1] = 0;
        dfsH(1);
        for (int i = 1; i <= N; ++i) cout << max(F[i], H[i]) << endl;
    }

    return 0;
}
/*
同类题目: Fiber-optic Network, ACM/ICPC 牡丹江2014, 牛客NC 127548
*/

```

例 2-42 【树形 DP；二次换根；树上流量】累计的度数 (Accumulation Degree, POJ 3585)

树形水系中，有 $N-1$ 条河道， N ($N \leq 2 \times 10^5$) 个交叉点。每条河道 $x-y$ 都有一个容量 $c(x,y)$ 。河道中单位时间流量不能超过河道容量。求每个交叉点作为根，到叶子结点的最大流量。

【代码实现】

```
// 二次扫描，换根
// 陈锋
#include <algorithm>
#include <cstdio>
#include <iostream>
```

```

#include <vector>

/*
记  $D(x)$  为以  $x$  为根的子树中，以  $x$  为源点从  $x$  发出的最大流量，则：
 $D(x) = \sum_{\text{Deg}(y) == 1} (x, y) : \min(D(y), c(x, y)), y \in \text{son}(x)$ 
任选  $x=1$  作为树根，DFS 一次，计算所有的  $D(u)$ 
记  $F(u)$  为  $u$  作为根到叶结点的最大流量。以 1 为根，则显然  $F(1)=D(1)$ 。
对于  $y$ ，记  $x=\text{pa}(y)$ ，如果  $\text{Degree}(x)=1$ ，则  $F(y)=D(y)+c(x, y)$ 
否则，从  $y$  流出的流量，就是  $D(y)+y \rightarrow x \rightarrow w$ 。
 $x \rightarrow y$  的部分为  $\min(D(y), c(x, y))$ 
那么  $x \rightarrow w$  就是  $F(x) - \min(D(y), c(x, y))$ 
 $F(y) = D(y) + \min(c(x, y), F(x) - \min(D(y), c(x, y)))$ 
*/
using namespace std;

const int MAXN = 2e5 + 4;
int N, F[MAXN], D[MAXN];
struct Edge {
    int v, w;
    Edge(int to = 0, int _w = 0) : v(to), w(_w) {}
};
vector<Edge> G[MAXN];
int dfs1(int u, int fa) {
    int& d = D[u];
    d = 0;
    for (size_t i = 0; i < G[u].size(); i++) {
        const Edge& e = G[u][i];
        if (e.v == fa) continue;
        int c = e.w;
        if (G[e.v].size() > 1) c = min(c, dfs1(e.v, u));
        d += c;
    }
    return d;
}

void dfs2(int u, int fa) {
    for (size_t i = 0; i < G[u].size(); i++) {
        const Edge& e = G[u][i];
        if (e.v == fa) continue;
        if (G[u].size() == 1)
            F[e.v] = D[e.v] + e.w;
        else
            F[e.v] = D[e.v] + min(e.w, F[u] - min(D[e.v], e.w));
        dfs2(e.v, u);
    }
}

int main() {
    int T;

```

```

scanf("%d", &T);
while (T--) {
    scanf("%d", &N);
    fill_n(D, N + 1, 0);
    for (int i = 0; i <= N; i++) G[i].clear();
    for (int i = 1, x, y, z; i < N; i++) {
        scanf("%d%d%d", &x, &y, &z);
        G[x].push_back(Edge(y, z)), G[y].push_back(Edge(x, z));
    }
    dfs1(1, -1);
    F[1] = D[1];
    dfs2(1, -1);
    int ans = *max_element(F + 1, F + N + 1);
    printf("%d\n", ans);
}
return 0;
}
/*
同类题目: Magic boy Bi Luo with his excited tree, HDU 5834
*/

```

例2-43 【树的重心】平衡的艺术 (Balancing Act, POJ 1655)

给出一棵包含 N ($1 \leq N \leq 20\ 000$) 个结点的树，结点编号从1开始。删除任何结点都会形成一个森林。定义结点的平衡度为删除这个结点后形成的森林中结点数最多的树的结点数。

例如，从如图2-9所示的树中删除结点4，会形成两棵树{5}和{1,2,3,6,7}，所以结点4的平衡度就是5。

计算整棵树上平衡度最小的结点。如果多个结点的平衡度相同，则输出编号最小的那个。

【代码实现】

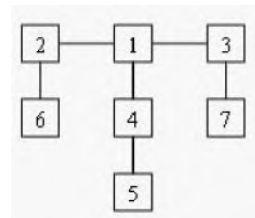


图2-9 平衡的艺术

```

// 陈锋
#include <cstdio>
#include <vector>
const int MAXN = 20000 + 4;
using namespace std;
int N, MaxS, Centroid;
vector<int> E[MAXN];
int dfs(int u, int fa) {
    int s = 1, max_s = -1;
    for (size_t i = 0; i < E[u].size(); i++) {
        int v = E[u][i];
        if (v == fa) continue;
        int sv = dfs(v, u);
        s += sv, max_s = max(max_s, sv);    // 统计最大的结点树的子结点数
    }
    max_s = max(max_s, N - s);            // 除子树 u 外的其他子树大小
    if (max_s < MaxS || (max_s == MaxS && Centroid > u))
        MaxS = max_s, Centroid = u;
    return s;
}

int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        scanf("%d", &N);
        for (int i = 1; i <= N; i++) E[i].clear();
        for (int i = 1, u, v; i < N; i++) {
            scanf("%d %d", &u, &v);
            E[u].push_back(v), E[v].push_back(u);
        }
        MaxS = MAXN, Centroid = 0;
        dfs(1, -1);
        printf("%d %d\n", Centroid, MaxS);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》9.4.2 节树的重心
同类题目:树的重心, CSP-S 2019 D2T3
*/

```

例2-44 【树的直径】牛的马拉松 (Cow Marathon, POJ 1985)

对于一棵包含 n 个结点的无根树，找到一条最长路径。换句话说，要找到两个点，使得它们之间的距离最远。

【代码实现】

```

// 陈锋
#include <algorithm>
#include <cstdio>
#include <iostream>
#include <vector>

const int MAXN = 1e6 + 4;
struct Edge {
    Edge(int _v, int _w) : v(_v), w(_w) {}
    int v, w;
};
using namespace std;
int N, M, ans;
vector<Edge> G[MAXN];
int D[MAXN];

void dfs(int u, int fa) {
    int &d = D[u], &dl = 0; // D到不同子树叶子的第1,2长路径
    d = 0;
    for (size_t i = 0; i < G[u].size(); i++) {
        const Edge& e = G[u][i];
        if (e.v == fa) continue;
        dfs(e.v, u);
        int nd = e.w + D[e.v];
        if (nd >= d)
            dl = d, d = nd;
        else if (nd > dl)
            dl = nd;
    }
    ans = max(ans, d + dl);
}

int main() {
    scanf("%d %d", &N, &M);
    char S[8];
    for (int i = 1, u, v, w; i <= M; i++) {
        scanf("%d %d %d %s", &u, &v, &w, S);
        G[u].push_back(Edge(v, w)), G[v].push_back(Edge(u, w));
    }
    ans = -1;
    dfs(1, -1);
    printf("%d\n", ans);
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》9.4.2节树的最长路径
注意：同一个结点往下的最长路径以及第二长路径的更新方法
*/

```

例2-45 【树上的动态规划；多个优化目标】放置街灯（Placing Lampposts, UVa 10859）

给你一个包含 n 个点、 m 条边（ $m < n \leq 1000$ ）的无向无环图，在尽量少的结点上放灯，使得所有边都能被照亮。每盏灯将照亮以它为端点的所有边。在灯的总数最小的前提下，被两盏灯同时照亮的边数应尽量大。

【代码实现】


```

// 刘汝佳
#include <cstdio>
#include <cstring>
#include <vector>
using namespace std;
const int NN = 1000 + 8, BASE = 2000;
vector<int> G[NN];           // 森林是稀疏的，这样节省空间，枚举相邻结点也更快
int Vis[NN][2], D[NN][2], N, M;

int dp(int i, int j, int f) { // DFS 的同时进行动态规划，f 是父结点，不存入状态里
    if (Vis[i][j]) return D[i][j];
    Vis[i][j] = 1;
    int& ans = D[i][j];

    // 放灯总是合法决策
    ans = BASE;           // 灯的数量加 1，x 加 BASE
    for (int k = 0; k < G[i].size(); k++)
        if (G[i][k] != f) // 这个判断非常重要！除父结点外的相邻结点才是子结点
            ans += dp(G[i][k], 1, i); // 注意，这些结点的父结点是 i
    if (!j && f >= 0) ans++; // 如果 i 不是根，且父结点没放灯，则 x 加 1

    if (j || f < 0) { // i 是根或者其父结点已放灯，i 才可以不放灯
        int sum = 0;
        for (int k = 0; k < G[i].size(); k++)
            if (G[i][k] != f) sum += dp(G[i][k], 0, i);
        if (f >= 0) sum++; // 如果 i 不是根，则 x 加 1
        ans = min(ans, sum);
    }
    return ans;
}

int main() {
    int T, a, b;
    scanf("%d", &T);
    while (T--) {
        scanf("%d%d", &N, &M);
        for (int i = 0; i < N; i++) G[i].clear();
        for (int i = 0, a, b; i < M; i++)
            scanf("%d%d", &a, &b), G[a].push_back(b), G[b].push_back(a);
        memset(Vis, 0, sizeof(Vis));
        int ans = 0;
        for (int i = 0; i < N; i++)
            if (!Vis[i][0]) ans += dp(i, 0, -1); // 新的一棵树的树根
        printf("%d %d %d\n", ans/BASE, M-ans%BASE, ans%BASE); // 从 x 计算 3 个整数
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 1.5.3 节例题 30
*/

```

例2-46 【货郎挑担问题（TSP）】匹萨派送（Hie with the Pie, POJ 3311）

有 $n+1$ 个城市，两两之间均有道路直接相连。给出两个城市 i 和 j 之间的道路长度 $D_{i,j}$ ，求一条可经过每个城市且仅经过一次，最后可回到起点的路线，使得经过的道路总长度最短。 $n \leq 10$ ，城市编号为 $0 \sim n$ 。

【代码实现】

```

// 陈锋
#include <algorithm>
#include <cstdio>
#include <cstring>
using namespace std;

const int NN = 10 + 1, INF = 0x3f3f3f3f;
int N, D[NN][NN], F[NN][1 << 12];           // 状态压缩

inline void floyd() {                       // D(i,j)->i 到 j 的最短距离
    for (int k = 0; k <= N; k++)
        for (int i = 0; i <= N; i++)
            for (int j = 0; j <= N; j++)
                D[i][j] = min(D[i][k] + D[k][j], D[i][j]); // i-j 的最短路
}
// F(i,s): 当前在 i, 还需要访问 s 中的点各一次
int dp(int i, int s) {
    int &f = F[i][s];
    if (s == 0) return f = D[i][0];         // 已达目的地
    if (f != -1) return f;
    f = INF;
    for (int j = 0; j <= N; j++) {
        if (s & (1 << j))                 // j: 下一个点
            f = min(f, D[i][j] + dp(j, s ^ (1 << j)));
    }
    return f;
}

int main() {
    while (scanf("%d", &N) == 1 && N) {
        memset(F, -1, sizeof(F)), memset(D, 0x3f, sizeof(D));
        for (int i = 0; i <= N; i++)
            for (int j = 0; j <= N; j++) scanf("%d", &D[i][j]);
        floyd(); // 结果 dp(0, (1,2,...,N-1))
        printf("%d\n", dp(0, (1 << (N + 1)) - 2));
    }
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》9.4.3 节货郎担问题(TSP)
同类题目: Nuts for nuts, UVa 10944
*/

```

例2-47 【集合动态规划；子集枚举】黑客的攻击 (Hacker' s Crackdown, UVa 11825)

假设你是一个黑客，侵入了一个有着 n ($1 \leq n \leq 16$) 台计算机（编号为 $0, 1, \dots, n-1$ ）的网络。一共有 n 种服务，每台计算机都运行着所有服务。对于任意一台计算机，你都可以选择一项服务，终止这台计算机和所有与它相邻的计算机的该项服务（如果其中一些服务已经停止，则这些服务继续处于停止状态）。你的目标是让尽量多的服务器完全瘫痪（即没有任何计算机运行该项服务），输出完全瘫痪的服务器的最大数量。

【代码实现】

```
// 陈锋
#include <algorithm>
#include <cstdio>
using namespace std;

const int NN = 16;
int P[NN], cover[1 << NN], f[1 << NN];
int main() {
    for (int kase = 1, n; scanf("%d", &n) == 1 && n; kase++) {
        for (int i = 0, m, x; i < n; i++) {
            scanf("%d", &m), P[i] = 1 << i;
            while (m--) scanf("%d", &x), P[i] |= (1 << x);
        }
        for (int S = 0; S < (1 << n); S++) {
            cover[S] = 0;
            for (int i = 0; i < n; i++)
                if (S & (1 << i)) cover[S] |= P[i];
        }
        f[0] = 0;
        int ALL = (1 << n) - 1;
        for (int S = 1; S < (1 << n); S++) {
            f[S] = 0;
            for (int S0 = S; S0; S0 = (S0 - 1) & S)
                if (cover[S0] == ALL) f[S] = max(f[S], f[S ^ S0] + 1);
        }
        printf("Case %d: %d\n", kase, f[ALL]);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 1.5.3 节例题 29
*/
```

例2-48 【状态压缩DP】玉米地 (Corn Fields, POJ 3254)

有一块长方形的牧场，被划分成 M 行、 N 列 ($1 \leq M, N \leq 12$)，每一格都是一块正方形的土地。FJ打算在牧场的某些格土地里种上美味的草，供他的奶牛们享用。遗憾的是，有些土地相当贫瘠，不能用来放牧。并且，奶牛们喜欢独占一块草地，于是FJ不会选择两块相邻的土地，即没有两块草地有公共边。当然，FJ还没有决定在哪些土地上种草。

作为一个好奇的农场主，FJ想知道，如果不考虑草地的总块数，那么一共有多少个种植方案可供他选择。当然，把新的牧场荒废，不在任何土地上种草，也算一个方案。请你帮FJ算一下这个总方案数。

【代码实现】

```

// 陈锋
#include <cstdio>
const int MOD = 100000000;
int B[13], St[377], F[13][377];
int main() {
    int S = 1 << 12, k = 0;
    for (int i = 0; i < (1 << 12); i++)
        if (!(i & (i << 1))) St[k++] = i;           // 合法的行状态
    St[k] = S;

    int M, N, t;
    scanf("%d%d", &M, &N);
    for (int i = 0; i < M; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &t), B[i] = (B[i] << 1) | !t;    // 荒地

    S = 1 << N;
    for (int i = 0; St[i] < S; i++) if (!(B[0] & St[i])) F[0][i] = 1; // 第0行
    for (int r = 1; r < M; r++)
        for (int i = 0; St[i] < S; i++) {                // 第 r-1 行状态
            if (B[r - 1] & St[i]) continue;
            for (int ri = 0; St[ri] < S; ri++)            // 第 r 行状态
                if (!(B[r] & St[ri]) && !(St[i] & St[ri])) // 不和上一行及当前行荒地冲突
                    (F[r][ri] += F[r - 1][i]) %= MOD;
        }

    int r = M - 1, ans = F[r][0];
    for (int i = 1; St[i] < S; i++)                        // 处理最后一行
        ans = (ans + F[r][i]) % MOD;
    printf("%d\n", ans);
    return 0;
}
/*
每行状态只影响下一行。
可以将所有合法的行状态都记录下来。(s & (s << 1)) == 0。荒地状态也记录下来。
F(r, s)，前 r 行，第 r 行状态已经决策成 s，总共多少种方案。
转移:  $F(i, s) = \sum F(i-1, ls)$ , s 和 ls 的 1 的位置不重复
一行行递推即可，最后答案是  $\sum F(M, s)$ 
注意：各种位运算技巧的运用。
同类题目：Dyslexic Gollum, ACM/ICPC Amritapuri 2012, UVa 1633
Locker, Tianjin 2012, UVa 1631
炮兵阵地, POJ 1185
*/

```

例2-49 【DAG上的DP；矩形嵌套问题】巴比伦塔（The Tower of Babylon, UVa 437）

有 n ($n \leq 30$) 种立方体，每种都有无穷多个。要求选一些立方体摞成一根尽量高的柱子（可以自行选择哪一条边作为高），使得每个立方体的底面长宽小于它下方立方体的底面长宽。

【代码实现】

```
// 刘汝佳  
#include <cstdio>
```

```

#include <cassert>
#include <cstring>
#include <algorithm>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;
const int maxn = 32;
struct Block{
    int x,y,z;
    int H[3], TP[3][2];
    void init(){
        H[0] = x;
        TP[0][0] = min(y,z), TP[0][1] = max(y,z);

        H[1] = y;
        TP[1][0] = min(x,z), TP[1][1] = max(x,z);

        H[2] = z;
        TP[2][0] = min(x,y), TP[2][1] = max(x,y);
    }
};
int n, D[maxn][3];
Block blocks[maxn];
int dp(int k, int h){ // 当前是第 i 个块, 第 h 条边作为其高度
    assert(0 <= h && h < 3);
    int& d = D[k][h];
    if(d != -1) return d;
    const Block& bk = blocks[k];
    d = bk.H[h];
    int x = bk.TP[h][0], y = bk.TP[h][1];
    _for(i, 0, n){
        const Block& b = blocks[i];
        _for(hi, 0, 3)
            if(b.TP[hi][0] < x && b.TP[hi][1] < y) d = max(d, bk.H[h] + dp(i, hi));
    }
    return d;
}

int main() {
    for(int t = 1; scanf("%d", &n) == 1 && n; t++){
        _for(i, 0, n) {
            Block& b = blocks[i];
            scanf("%d%d%d", &(b.x), &(b.y), &(b.z));
            b.init();
        }
        memset(D, -1, sizeof(D));
        int ans = -1;
    }
}

```



```

    _for(i, 0, n) _for(hi, 0, 3) ans = max(ans, dp(i, hi));
    printf("Case %d: maximum height = %d\n", t, ans);
}
return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题 9-2
同类题目: A Spy in the Metro, ACM/ICPC World Finals 2003, UVa 1025
*/

```

例 2-50 【斜率优化 DP】任务安排 (Batch Scheduling, IOI 2002, 牛客 NC 51204)

有 N 个任务排成一个序列，在一台机器上等待执行，且它们的顺序不得改变。机器会把这 N 个任务分成若干批，每批包含连续的若干个任务。从时刻 0 开始，任务被分批加工，执行第 i 个任务所需的时间是 T_i 。另外，在每批任务开始前，机器需要 S 的启动时间，故执行一批任务所需的时间是启动时间 S 加上每个任务所需的时间。

一个任务执行后，将在机器中短暂等待，直至该批任务全部执行完毕。也就是说，同一批任务将在同一时刻完成。每个任务的费用是它的完成时刻乘以一个费用系数 C_i 。

请为机器规划一个分组方案，使得总费用最小。

【代码实现】

```

// 陈锋
#include<bits/stdc++.h>

using namespace std;
typedef long long LL;
const int NN = 10000 + 4;
int N, S, T[NN], C[NN], Q[NN], G[NN], D[NN];
inline double f(int k, int i) {return (double) (D[k] - D[i]) / (Q[k] - Q[i]); }

int main() {
    scanf("%d%d", &N, &S);
    Q[N + 1] = G[N + 1] = D[N + 1] = 0;
    for (int i = 1; i <= N; i++) scanf("%d%d", &(T[i]), &(C[i]));
    deque<int> q;
    q.push_back(N + 1);
    for (int i = N; i >= 1; i--) {
        Q[i] = T[i] + Q[i + 1], G[i] = C[i] + G[i + 1];
        while (q.size() >= 2 && f(q[0], q[1]) < (double) G[i])
            q.pop_front();
        D[i] = D[q[0]] + (S + Q[i] - Q[q[0]]) * G[i];
        while (q.size() >= 2 && f(q[q.size() - 2], q.back()) > f(q.back(), i))
            q.pop_back();
        q.push_back(i);
    }
    printf("%d\n", D[1]);
    return 0;
}
/*
算法分析请参考: http://www.ioi2002.or.kr/eng/tasks/batch-handout.pdf
*/

```

例2-51 【最优三角剖分】最大面积最小的三角剖分 (Minimax Triangulation, ACM/ICPC NWERC 2004, UVa 1331)

三角剖分是指用不相交的对角线把一个多边形分成若干个三角形, 如图2-10所示是一个六边形几种不同的三角剖分方案。

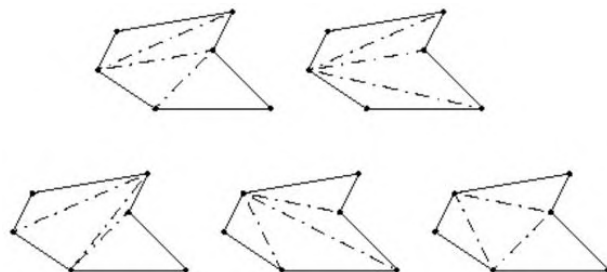


图2-10 六边形的不同三角剖分

输入一个简单的 m ($2 < m < 50$) 边形, 寻找一种三角剖分方案, 使得其中最大三角形的面积最小。输出最大三角形的面积。在图2-10所示的5个方案中, 左下角的方案最优。

【代码实现】

```
// 刘汝佳
#include<bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;

const double eps = 1e-10;
int dcmp(double x) {
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
};

typedef Point Vector;

Vector operator+(const Vector& A, const Vector& B)
{ return Vector(A.x + B.x, A.y + B.y); }
Vector operator-(const Point& A, const Point& B)
{ return Vector(A.x - B.x, A.y - B.y); }
Vector operator*(const Vector& A, double p)
{ return Vector(A.x * p, A.y * p); }
bool operator<(const Point& a, const Point& b)
```

```

{ return a.x < b.x || (a.x == b.x && a.y < b.y); }
bool operator==(const Point& a, const Point& b)
{ return dcmp(a.x - b.x) == 0 && dcmp(a.y - b.y) == 0; }
double Dot(const Vector& A, const Vector& B)
{ return A.x * B.x + A.y * B.y; }
double Cross(const Vector& A, const Vector& B)
{ return A.x * B.y - A.y * B.x; }
double Length(Vector A) { return sqrt(Dot(A, A)); }
bool SegmentProperIntersection(const Point& a1, const Point& a2,
                                const Point& b1, const Point& b2) {
    double c1 = Cross(a2 - a1, b1 - a1), c2 = Cross(a2 - a1, b2 - a1),
           c3 = Cross(b2 - b1, a1 - b1), c4 = Cross(b2 - b1, a2 - b1);
    return dcmp(c1) * dcmp(c2) < 0 && dcmp(c3) * dcmp(c4) < 0;
}

bool OnSegment(const Point& p, const Point& a1, const Point& a2) {
    return dcmp(Cross(a1 - p, a2 - p)) == 0 && dcmp(Dot(a1 - p, a2 - p)) < 0;
}

typedef vector<Point> Polygon;

int isPointInPolygon(const Point& p, const Polygon& poly) {
    int n = poly.size();
    int wn = 0;
    for (int i = 0; i < n; i++) {
        const Point& p1 = poly[i];
        const Point& p2 = poly[(i + 1) % n];
        if (p1 == p || p2 == p || OnSegment(p, p1, p2)) return -1; // 在边界上
        int k = dcmp(Cross(p2 - p1, p - p1));
        int d1 = dcmp(p1.y - p.y);
        int d2 = dcmp(p2.y - p.y);
        if (k > 0 && d1 <= 0 && d2 > 0) wn++;
        if (k < 0 && d2 <= 0 && d1 > 0) wn--;
    }
    if (wn != 0) return 1; // 内部
    return 0; // 外部
}

bool isDiagonal(const Polygon& poly, int a, int b) {
    int n = poly.size();
    for (int i = 0; i < n; i++)
        if (i != a && i != b && OnSegment(poly[i], poly[a], poly[b]))
            return false; // 中间不能有其他点
    for (int i = 0; i < n; i++)
        if (SegmentProperIntersection(poly[i], poly[(i + 1) % n], poly[a], poly[b]))
            return false; // 不能和多边形的边规范相交
    Point midp = (poly[a] + poly[b]) * 0.5;
    return (isPointInPolygon(midp, poly) == 1); // 整条线段在多边形内
}

```

```

const int MAXM = 50 + 2;
int M;
double D[MAXM][MAXM];
double dp(const Polygon& poly, int i, int j) {
    double& d = D[i][j];
    if (d > -1) return d;
    if (i + 1 == j) return d = 0;
    d = DBL_MAX;
    if (!(i == 0 && j == M - 1) && !isDiagonal(poly, i, j)) return d = DBL_MAX;
    _for(k, i + 1, j) {
        double m = max(dp(poly, i, k), dp(poly, k, j));
        // (i,j,k)-面积
        m = max(m, fabs(Cross(poly[i] - poly[j], poly[i] - poly[k])) / 2.0);
        d = min(d, m);
    }
    return d;
}

// 输入一个M边形，寻找其最大三角形的面积最小的三角剖分方案，输出最大三角形的面积
double solve(const Polygon& poly) {
    _for(i, 0, M) _for(j, 0, M) D[i][j] = -1;
    return dp(poly, 0, M - 1);
}

int main() {
    int T;
    ios::sync_with_stdio(false), cin.tie(0);
    cin >> T;
    double x, y;
    Polygon p;
    while (T--) {
        cin >> M;
        p.clear();
        _for(i, 0, M) cin >> x >> y, p.emplace_back(x, y);
        double ans = solve(p);
        printf("%.11f\n", ans);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 9-11
同类题目：Triangle Count Sequences of Polygon Triangulations, ACM/ICPC Greater NY 2013
*/

```

例2-52 【单调队列优化DP】纸币（Banknotes, POI 2005，牛客 NC 50532）

Byteotian Bit Bank (BBB) 拥有一套先进的货币系统，该系统共有 n 种不同面值的硬币，面值分别为 $b_1, b_2, \dots, b_n$ 。已知每种硬币都有一定的数量限制，问：想要凑出面值 k ，最少要用多少个硬币？

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
const int NN = 210, MM = 20010, INF = 0x3f3f3f3f;
typedef pair<int, int> IPair;
int N, A[NN], C[NN], M, F[MM];
struct Item {int fk, k; };
int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    cin >> N;
    memset(F, 0x3f, sizeof(F)); F[0] = 0;
    for (int i = 1; i <= N; ++i) cin >> A[i];
    for (int i = 1; i <= N; ++i) cin >> C[i];
    cin >> M;
    deque<Item> Q;
    for (int i = 1; i <= N; ++i) { // 考虑 F[1...i]
        int a = A[i];
        for (int r = 0; r < a; ++r) { // r = k%M
            Q.clear();
            /*
                f(i, r + xa) = min(f(i - 1, r + ya) + x - y), y < x < y + C[i]
                f(i, r + xa) - x = min(f(i - 1, r + ya) - y)
            */
            for (int k = 0; k * a + r <= M; ++k) {
                int x = k * a + r;
                while (!Q.empty() && k - Q.front().k > C[i]) Q.pop_front();
                while (!Q.empty() && F[x] - k <= Q.front().fk) Q.pop_back();
                Q.push_back({F[x] - k, k});
                F[x] = min(F[x], Q.front().fk + k);
            }
        }
    }
    cout << F[M] << endl;
    return 0;
}
/*
同类题目: UVa 12170 Easy Climb
*/
```

例 2-53 【数位统计】统计问题 (The Counting Problem, ACM/ICPC上海2004, UVa 1640)

给出整数 a , b , 统计 a 和 b (包含 a 和 b) 之间的整数中, 数字 $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$ 分别出现了多少次。 $1 \leq a, b \leq 10^8$ 。注意, a 有可能大于 b 。

【代码实现 (C++11)】

```
// 陈锋  
#include <bits/stdc++.h>  
using namespace std;
```

```

#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;
typedef valarray<int> TRes;

int P10[9]; // P10[i] = 10^i
TRes solve(int N) { // 计算 0~N 的每个值
    TRes ans(0, 10);
    if (N == 0) {
        ans[0] = 1;
        return ans;
    }
    auto S = to_string(N);
    int W = S.size();
    _for(k, 1, W) { // ***** 宽度为 k 的数字
        if (k == 1) { // 0~9
            ans += 1;
            continue;
        }
        _rep(d, 1, 9) ans[d] += P10[k - 1]; // d*****
        ans += 9 * P10[k - 2] * (k - 1); // 9*(10^(k-1))*(k-1)/10
    }
    _for(i, 0, W - 1) {
        int sd = i == 0 ? 1 : 0;
        _for(d, sd, S[i] - '0') { // 考虑-----d***** d<S[i]
            int l = W - i - 1; // -----d***** 中*的个数
            _for(j, 0, i) ans[S[j] - '0'] += P10[l]; // -----d***** 中的 -
            ans[d] += P10[l]; // -----d***** 中的 d
            ans += P10[l - 1] * 1; // -----d***** 中的*
        }
    }

    _rep(d, 0, S[W - 1] - '0') { //*****d
        _for(j, 0, W - 1) ans[S[j] - '0']++;
        ans[d]++;
    }

    return ans;
}

TRes solve(int a, int b) {
    if (a > b) swap(a, b);
    const TRes &ta = solve(a - 1), &tb = solve(b);
    return tb - ta;
}

int main() {
    P10[0] = 1;
    _for(i, 1, 9) P10[i] = 10 * P10[i - 1];
}

```



```

for (int a, b; scanf("%d%d", &a, &b) == 2 && a && b; ) {
    const auto& ans = solve(a, b);
    _for(i, 0, 10) printf("%d%s", ans[i], i == 9 ? "\n" : " ");
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题10-22
注意：本题使用了 C++11 中新引入的数据结构 valarray，来简化对一个数组中所有元素的同时操作
*/

```

例2-54 【数位统计】Windy数（牛客NC 20268）

Windy定义了一种Windy数：不含前导零且相邻两个数字之差至少为2的正整数倍。询问在 A 和 B 之间（包括 A, B ）总共有多少个Windy数？

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (int)(b); ++i)

int F[20][20];          // F[i][j]表示i位数，最高位为j的windy数的个数
void init() {
    _for(i, 0, 10) F[1][i] = 1; // 1位数
    _rep(i, 2, 10) _for(j, 0, 10) _for(k, 0, 10) if (abs(j - k) >= 2) {
        F[i][j] += F[i - 1][k];
    }
}

int solve(int x) {      // 小于x的windy数
    int ans = 0, len = 0, b[20];
    while (x > 0) b[++len] = x % 10, x /= 10; // 从右到左解析
    b[len + 1] = -1;
    _for(i, 1, len) _for(j, 1, 10) ans += F[i][j]; // 比x短的数字
    _for(i, 1, b[len]) ans += F[len][i];          // 位数相同，但是高位小于b的高位
    // 目标数字从右数第i+1开始和x都相同，但是i位比x小，且前面部分符合windy数的规则
    for (int i = len - 1; i > 0; i--) {
        _for(j, 0, b[i]) if (abs(j - b[i + 1]) >= 2) ans += F[i][j];
        if (abs(b[i] - b[i + 1]) < 2) break;
    }
    return ans;
}

typedef long long LL;
int main() {
    init();
    ios::sync_with_stdio(false), cin.tie(0);
    for (int x, y; cin >> x >> y;)

        cout << solve(y + 1) - solve(x) << endl;
    return 0;
}
/*
同类题目：Sum of Log, ACM/ICPC 上海2020, Codeforces Gym 102900C
*/

```

例2-55 【轮廓线动态规划；覆盖模型】铺放骨牌（Tiling Dominoes, UVa 11270）

用 1×2 骨牌覆盖 $n \times m$ ($n, m \leq 100$) 棋盘，有多少种方法？如图2-11所示，是一种铺放方法。

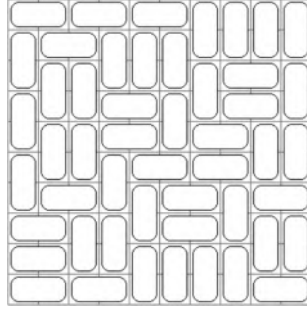


图2-11 铺放骨牌

【代码实现】

```

// 刘汝佳
#include <stdio>
#include <string>
#include <algorithm>
using namespace std;
int n, m, cur;

const int maxn = 15;
long long d[2][1<<maxn], memo[maxn*maxn][maxn*maxn];

void up(int a, int b) {
    if(b & (1<<m)) d[cur][b^(1<<m)] += d[1-cur][a];
}

long long solve(int n, int m) {
    memset(d, 0, sizeof(d));
    cur = 0;
    d[0][(1<<m)-1] = 1;
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++) { // 枚举当前要算的阶段
            cur ^= 1;
            memset(d[cur], 0, sizeof(d[cur]));
            for(int k = 0; k < (1<<m); k++) { // 枚举上个阶段的状态
                up(k, k<<1);
                if(i && !(k & (1<<m-1))) up(k, (k<<1)^(1<<m)^1);
                if(j && !(k & 1)) up(k, (k<<1)^3);
            }
        }
    return d[cur][(1<<m)-1];
}

int main() {
    memset(memo, -1, sizeof(memo));
    while(scanf("%d%d", &n, &m) == 2) {
        if(n < m) swap(n, m);
        if(memo[n][m] < 0) memo[n][m] = solve(n, m);
        printf("%lld\n", memo[n][m]);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 6.1 节例题 1
同类题目：UVa 1214, Manhattan Wiring
*/

```

例2-56 【扫描线】最大子矩阵（City Game, SEERC 2004, POJ 1964）

给定一个 $m \times n$ ($1 \leq m, n \leq 1000$) 的矩阵，其中一些格子是空地 (F)，其他格子是障碍 (R)。找出一个全部由 F 组成的面积最大的子矩阵，输出其面积乘以3后的结果。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cstdio>
using namespace std;

const int maxn = 1000;
int mat[maxn][maxn], up[maxn][maxn], left[maxn][maxn], right[maxn][maxn];
int main() {
    int T, m, n;
    scanf("%d", &T);
    while (T--) {
        scanf("%d%d", &m, &n);
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++) {
                int ch = getchar();
                while (ch != 'F' && ch != 'R') ch = getchar();
                mat[i][j] = ch == 'F' ? 0 : 1;
            }

        int ans = 0;
        for (int i = 0; i < m; i++) { // 从上到下逐行处理
            int lo = -1, ro = n;
            for (int j = 0; j < n; j++) { // 从左到右扫描, 维护 up 和 left
                int &u = up[i][j], &l = left[i][j];
                if (mat[i][j] == 1)
                    u = l = 0, lo = j;
                else {
                    u = i == 0 ? 1 : up[i - 1][j] + 1;
                    l = i == 0 ? lo + 1 : max(left[i - 1][j], lo + 1);
                }
            }
            for (int j = n - 1; j >= 0; j--) { // 从右到左扫描, 维护 right 并更新答案
                int &r = right[i][j];
                if (mat[i][j] == 1)
                    r = n, ro = j;
                else {
                    r = i == 0 ? ro - 1 : min(right[i - 1][j], ro - 1);
                    ans = max(ans, up[i][j] * (r - left[i][j] + 1));
                }
            }
        }
        printf("%d\n", ans * 3); // 输出最大面积乘以 3 后的结果
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 1.3 节例题 22
同类题目: Cricket Field, ACM/ICPC NEERC 2002, UVa 1312
*/

```

本节例题列表

本章讲解的例题及其囊括的知识点，如表2-4所示。

表2-4 动态规划算法例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 2-31	POJ 3624	Charm Bracelet	01 背包；滚动数组优化	陈锋
例 2-32	UVa 1220	Party at Hali-Bula	树的最大独立集	陈锋
例 2-33	POJ 1458	Common Subsequence	最长公共子序列问题（LCS）	陈锋
例 2-34	POJ 1836	Alignment	最长上升子序列问题（LIS）	陈锋
例 2-35	UVa 10635	Prince and Princess	LCS；可转化为 LIS	陈锋
例 2-36	HDU 1231	最大连续子序列	最大连续和	陈锋
例 2-37	POJ 1651	Multiplication Puzzle	矩阵链乘	陈锋
例 2-38	UVa 10003	Cutting Sticks	区间切割 DP	陈锋
例 2-39	UVa 1626	Brackets Sequence	括号序列	陈锋
例 2-40	UVa 11584	Partitioning by Palindromes	字符串划分问题；回文预处理	陈锋
例 2-41	HDU 2196	Computer	树形 DP；二次换根	陈锋
例 2-42	POJ 3585	Accumulation Degree	树形 DP；二次换根；树上流量	陈锋
例 2-43	POJ 1655	Balancing Act	树的重心	陈锋
例 2-44	POJ 1985	Cow Marathon	树的直径	陈锋
例 2-45	UVa 10859	Placing Lampposts	树上的动态规划；多个优化目标	刘汝佳
例 2-46	POJ 3311	Hie with the Pie	货郎挑担问题（TSP）	陈锋
例 2-47	UVa 11825	Hacker's Crackdown	集合动态规划；子集枚举	陈锋
例 2-48	POJ 3254	Corn Fields	状态压缩 DP	陈锋
例 2-49	UVa 437	The Tower of Babylon	DAG 上的 DP；矩形嵌套问题	刘汝佳
例 2-50	NC 51204	Batch Scheduling	斜率优化 DP	陈锋
例 2-51	UVa 1331	Minimax Triangulation	最优三角剖分	刘汝佳
例 2-52	NC 50532	Banknotes	单调队列优化 DP	陈锋
例 2-53	UVa 1640	The Counting Problem	数位统计	陈锋
例 2-54	NC 20268	Windy 数	数位统计	陈锋
例 2-55	UVa 11270	Tiling Dominoes	轮廓线动态规划；覆盖模型	刘汝佳
例 2-56	POJ 1964	City Game	扫描线	刘汝佳